

JobPilot: Smart Job Matcher and Resume Builder

1. Objective, User Flow, and Architecture

JobPilot is a job matching and resume-support application for the BAX-423 final project. It implements the required workflow from job ingestion to deduplication, profile intake, matching, ranking, adaptive feedback, CSV export, analytics, deployment, and API-gated resume generation. The default grading path is reproducible without private job-search or LLM API keys: matching runs from a packaged 50,000-row offline job snapshot, local embeddings, ANN/sklearn retrieval, hard filters, and a deployed FastAPI app. Optional live refresh and Gemini resume/profile reading are API-key paths, not dependencies for the no-key matching workflow.

The user flow is direct. A user selects a demo persona or enters a custom profile/resume, optionally parses it with the local reader or Gemini, reviews executable filters, and runs matching. The result page returns ranked job cards with title, company, location, salary text, employment type, match-strength labels, matched-skill chips, why-this-match reasons, watch-outs, expandable details, feedback buttons, CSV download, analytics, and resume generation when an LLM key is connected. In the hosted app, a Gemini key entered for profile reading is kept only in browser session storage and can unlock resume generation for that same browser session.

The web application is built with FastAPI, Jinja templates, plain CSS, local services, and a processed CSV-backed data layer. The main routes support profile parsing, matching, feedback capture, reranking from prior session events, resume generation, analytics, optional live refresh preview, and top-job CSV download. The architecture keeps ANN retrieval, hard filters, user-visible explanation fields, session adaptation, and learned-reranker experiments separate. This matters because the app must explain recommendations without exposing hidden backend scores, raw embeddings, raw job descriptions, private diagnostics, or unreviewed evidence spans as model inputs.

Core requirement coverage:

Requirement	JobPilot implementation
Job ingestion and deduplication	50,000-row processed snapshot, ingestion report, data dictionary, duplicate removal, source-field normalization
Profile intake and skill matching	Demo personas, manual structured profile, PDF/profile text parsing, local reader, optional Gemini reader
Embedding and ANN retrieval	Dense job/profile representations with ANN-style retrieval and sklearn fallback
Ranking and re-ranking	JobRanker hard filters, 200-candidate reservoir, learned-active rerank when artifact loads, R0+safe-sidecar fallback
Adaptive learning	Accept/reject/skip logging plus session-level feedback memory; offline simulated feedback and persona-agent studies
Delivery	Cloud Run app, CSV export, market analytics, job explanations, API-gated Word resume generation

2. Data Pipeline and BAX-423 Techniques

The processed runtime dataset contains 50,000 job rows. The latest ingestion run scanned 51,636 candidate records, ingested 50,000 rows, removed exact duplicates, and completed in about 44 seconds from the materialized candidate cache. The snapshot is built from a large offline job corpus plus saved current-posting payloads. It preserves source-backed fields for job title, company, location, salary text, description, skills, industries, posting dates, links, and direct-apply metadata. Raw API payloads, private keys, local caches, runtime sessions, and old package ZIPs are excluded from the submission package.

Technique 1 is streaming-style ingestion with data quality controls. The ingestion workflow audits schema coverage, normalizes fields, removes unusable rows, keeps source-backed values where possible, and writes durable reports. This supports a reproducible grading path while still allowing optional current-job refreshes when credentials are available.

Technique 2 is hash/Bloom-assisted exact deduplication. Compact duplicate checks reduce lookup work, but exact verification is still required before a row is removed as a duplicate. This makes deduplication scalable without turning approximate membership checks into irreversible data-loss decisions.

Technique 3 is dense embeddings and ANN-style retrieval. Profile and job text are represented with sentence-transformer embeddings, using all-MiniLM-L6-v2 in the current 50,000-row benchmark. The retrieval service uses ANN-style retrieval with a sklearn fallback when FAISS is unavailable. A lexical TF-IDF retriever is used only as a comparison baseline, not as the main matching method.

Technique 4 is multi-stage ranking with explainability and adaptive feedback. Retrieved jobs pass through authoritative hard filters, a 200-row eligible reservoir, runtime-safe feature construction, reranking, visible dealbreaker suppression, quality gating, deterministic tie-breaking, and UI explanation builders. Session feedback applies only to later requests: same-job rejects suppress the job in the session, same-company rejects/skips penalize exposure, and accepts can boost same company or overlapping skills. The current-row interaction action is never used as a same-row model predictor.

Candidate-generation benchmark evidence:

Method	Strict P@10	Strict dealbreaker violation	Latency
Lexical TF-IDF comparison baseline	0.35	0.65	0.0344s
Sentence-transformer embedding / ANN retrieval	0.55	0.45	0.0639s

This table compares candidate generators before the full app applies hard filters, scoring, explanations, and adaptive reranking. It shows that sentence-transformer retrieval improves strict Precision@10 over lexical retrieval on the four assignment personas. The deployed app should be judged on the full matching path below, where hard filters remain authoritative.

3. Persona Evaluation and Feedback Simulation

The four assignment personas are used as source-provided evaluation anchors. Pass means the app returned a complete Top 10 and the latest deterministic persona-utility smoke test found no hard-filter violations while producing high accept concentration in visible recommendations. These are app-validation results, not real user feedback.

Persona	Assignment check	Latest smoke result	Status
Aisha	ML-related roles; avoid Senior/Staff/Principal, 5+ years, defense/military	mean@10 0.7620; accept@10 0.90; hard-filter violations 0	Pass
Marcus	Entry analytics/BI; avoid 3+ years, contract/temp, unpaid roles	mean@10 0.8261; accept@10 1.00; hard-filter violations 0	Pass
Kenji	ML/AI new grad; sponsorship needed; avoid contract/temp and no-sponsor jobs	mean@10 0.8264; accept@10 0.90; hard-filter violations 0	Pass
Priya	Senior ML infra/MLOps; no Junior roles or tiny startups; NYC/remote/US preference	mean@10 0.7970; accept@10 1.00; hard-filter violations 0	Pass

The main remaining limitation is evidence quality. Public postings often omit sponsorship policy, company size, exact work-authorization text, or precise location eligibility. The app surfaces those gaps as watch-outs and treats them as review context rather than silently converting missing evidence into confirmed eligibility.

For adaptive learning, JobPilot includes deterministic feedback simulation and a stricter persona-agent interaction study. The stricter study uses 15 simulated personas, 25 jobs per persona, and 3 rounds, for 1,125 interactions. Simulated users can only see the same UI-projected job-card fields that a real user would see. They cannot cite backend score components, vector similarity, raw sidecar fields, raw embeddings, hidden boosts, private diagnostics, or fields not projected into the UI. Each interaction records outcome_label separately from behavior metadata: accept=1.0, skip=0.5, reject=0.0. Expand/collapse behavior and rationale are audit metadata rather than gold labels.

Adaptive-learning summary:

Metric	Round 0	Round 2	Change
Interactions	375	375	0
Accept rate @10	0.4533	0.6600	+0.2067
Mean outcome @10	0.6167	0.7733	+0.1566
Reject rate @10	0.2200	0.1133	-0.1067
Validator pass rate	1.0000	1.0000	0

Across all rounds, the interaction log contains 393 accepts, 374 skips, and 358 rejects. The validator pass rate is 100%, with zero hidden-field violations and zero dealbreaker violations in the interaction validator. These outputs are simulated offline traces, not real user feedback and not gold-label training data.

4. Learned Reranker and Runtime Policy

The learned-reranker experiment converts the interaction log into a safe, low-dimensional feature table with 1,125 rows and 44 columns. Allowed model features include rank shown, salary-listed flags, salary bucket, location match, remote/preferred-location flag, employment type match, years-required bucket, seniority warning flag, work-authorization visibility flag, match-strength label encoding, matched-skill count, reason/watch-out counts, UI-projection completeness, persona intent flags, and previous-round feedback history. It deliberately excludes free-text rationale, raw job descriptions, raw embeddings, ANN vector data, backend-only diagnostics, raw sidecar rows, raw LLM evidence spans, evidence-adjustment internals, and current-row interaction action as a same-row predictor.

The current runtime is Phase 2.18J aligned: ANN/sklearn retrieval remains unchanged, JobRanker hard filters remain unchanged, the eligible reservoir is capped at 200, and display Top N remains the UI setting. If the EBM artifact loads, learned_active scores eligible candidates and then applies prior-session feedback and safe sidecar overlays. If the artifact cannot load or explicit fallback mode is selected, the app uses the R0 + safe-sidecar fallback policy, not the older production/rule rerank as the default. Old rule rerank remains available only as a diagnostic comparator.

Sidecar features are handled conservatively. Company-size, sponsorship, LCA/H-1B activity, and reviewed LLM overlay fields are converted into low-dimensional safe features and diagnostics. The current EBM artifact was not retrained on the newly added sidecar fields, so those fields are shadowed for future retraining and can only affect runtime through conservative overlay rules. LCA is treated only as historical employer filing activity, never job-level sponsorship truth. The reviewed LLM overlay is partial review-only evidence, never a gold label.

The model experiment compared the current baseline, a rule feedback baseline, and an EBM reranker:

Method	NDCG@10	Accept@10	Mean outcome @10	Interpretation
Current matching baseline	0.8708	0.6467	0.7200	Reference
Rule feedback baseline	0.8779	0.6667	0.7400	Strong simple comparator
EBM reranker	0.8591	0.6333	0.7433	Learned prototype with guardrails

The EBM improved mean outcome@10 but lagged the rule baseline on NDCG@10 and accept@10 in the offline comparison. For the course prototype, this is why the deployed system keeps the EBM behind hard filters, quality gates, deterministic tie-breaking, fallback metadata, and an R0+safe-sidecar fallback. The project demonstrates a learned reranker path without claiming real-user feedback or retraining inside /match.

5. Deliverables, Deployment, and Limitations

The submission contains runnable source code, requirements, Docker/Cloud Run configuration, prompt documentation, processed data files, benchmark reports, persona-agent logs, safe feature manifests, model reports, and this brief. The app supports ranked recommendations, user-facing explanations, feedback controls, analytics, CSV download with job details, and API-gated resume generation.

The deployed Cloud Run URL is <https://jobpilot-816874777792.us-west1.run.app>. The service uses the packaged processed dataset for a stable no-key demo path. The latest deployed revision was validated after deployment: the first match after a new revision can be slow because Cloud Run cold-loads ranking artifacts, while warm /match requests were measured at about 2.5-2.8 seconds with the 200-candidate MVP reservoir. Cloud Run is configured with min instances to reduce but not eliminate cold starts.

Local verification is part of the submission boundary. The current test suite and final readiness checks were used to validate ingestion, ranking, feedback, and app behavior. Packaging warnings are about files that should not be manually included in the Canvas ZIP, such as private key folders, raw data, runtime storage, old ZIPs, and local SQLite state. These are excluded by ignore rules and are not needed for the no-key grading workflow.

Production boundaries are explicit. Feedback simulations, persona-agent traces, sidecar overlays, and learned-model experiments are course-project evidence only. They are useful for auditing adaptive-learning behavior and future planning, but they do not replace ANN retrieval, embedding text, Phase 1 ingestion, or JobRanker hard filters. Salary, sponsorship, company size, and seniority evidence remain sparse or noisy in public postings. Optional live refresh, Gemini profile reading, and LLM resume drafting are API-key paths; the default matching, feedback, CSV, and analytics workflow remains offline and reproducible.